

**ADVANCED SYSTEMS LABORATORY
DEFENCE RESEARCH AND DEVELOPMENT ORGANISATION**



INTERNSHIP REPORT

On

Analysis and Implementation of FFT Algorithms

Submitted by

ROOPESH O R (20323085)

*Submitted in partial fulfillment of the requirements for the
B.Tech in Electronics and Communication Engineering*

Under the Guidance of
Venket Reddy G, Sc 'F'

Internship Duration:
May 18 2026 - June 15 2026

**DIVISION OF ELECTRONICS ENGINEERING
SCHOOL OF ENGINEERING
COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY
KOCHI - 682022**

July 2026

Acknowledgment

The satisfaction that accompanies the successful completion of this project would be incomplete without the mention of the people who made it possible, without whose constant guidance and encouragement would have made efforts go in vain. We consider ourselves privileged to express gratitude and respect towards all those who guided us through the completion of this project. It is our privilege to express our sincerest regards to our project coordinator, Mrs. Krishnapriya C V, Mrs. Mumtaz P Ali and Mr. Unni A M for their valuable inputs, able guidance, encouragement, whole-hearted cooperation and constructive criticism throughout the duration of our project. We deeply express our sincere thanks to our Head of Department, Dr. Deepa Sankar for encouraging and allowing us to present the project at our department premises. We take this opportunity to thank all our lecturers who directly or indirectly helped our project. We pay our respects and love to our parents and all other family members and friends for their love and encouragement throughout our career. Last but not the least we express our thanks to our friends for their cooperation and support.

Table of contents

List of Figures	ii
List of Tables	iii
1. Overview of internship	1
1.1. Introduction	1
1.2. Artix-7	1
2. Scalable methods	2
2.1. Method used in analysis	2
2.2. Cooley-Tukey: Radix-2 DIT	2
2.2.1. Implementation	4
2.2.2. Simulation	6
2.3. Radix-2 Multi Path Delay Commutator (R2MDC)	7
2.3.1. Implementation	7
2.3.2. Simulation result	9
2.4. Radix-2 Single Path Delay Commutator (R2SDC)	10
2.4.1. Implementation	10
2.4.2. Simulation result	12
3. Specialized methods	13
3.1. Winograd	13
3.1.1. Derivation	13
4. Comparison	17

List of Figures

Figure 2.1	Steps involved in Cooley-Tukey algorithm	3
Figure 2.2	Signal flow graph of 8-point radix-2 DIT-FFT	4
Figure 2.3	Working of FSM in Radix-2 DIT FFT	5
Figure 2.4	Block diagram of Radix-2 DIT FFT implementation	6
Figure 2.5	Timing diagram of output of 4 point Radix-2 DIT-FFT	6
Figure 2.6	Signal flow graph of 4-point radix-2 DIF-FFT	7
Figure 2.7	Steps in R2MDC algorithm with $N = 4$ case	8
Figure 2.8	Schematic of R2MDC implementation	9
Figure 2.9	Inside a single R2MDC Stage. All modules with wedge is clocked	9
Figure 2.10	Simulation result for 4 point complex test signal	9
Figure 2.11	Schematic of R2SDC implementation	10
Figure 2.12	Inside a single R2SDC Stage. All modules with wedge is clocked	10
Figure 2.13	Steps involved in a 4 point R2SDC algorithm	11
Figure 2.14	Simulation result for 4 point complex test signal	12

List of Tables

Chapter 1:

Overview of internship

1.1 Introduction

In Digital Signal Processing (DSP), the need to express a time domain signal to frequency domain often comes in various settings. For this Fourier Transform (FT) is widely employed. However FT is computationally expensive. In past century various algorithms have been proposed to compute FT of signals with limited compute power and memory. Even to this date, various forms of FT algorithms have been proposed. These set are now commonly called Fast Fourier Transform (FFT) algorithms.

My internship is involved in implementing few of these algorithms using SystemVerilog suitable for FPGA. The architecture and operation of these algorithms are detailed in the following chapter, as well as the hardware requirements and some performance aspects are also discussed along with them. In the last chapter, a comparative analysis of all algorithms are outlayed.

For analysis purpose Artix-7 FPGA is used in all the implementation. Operating frequency of 100 MHz is targeted in all algorithms. However as we will some algorithm cannot achieve this while others can go beyond this point.

1.2 Artix-7

Chapter 2:

Scalable methods

2.1 Method used in analysis

The initial development, simulation and debugging was done using verilator due to its ease of use. Synthesis and implementation were carried out in Vivado. To automate the process of generating implementation reports for multiple point size FFTs, a TCL script was created and vivado was run from terminal with the script, with output to a file. Hardware utilisation, nets, routing summary, timing and power characteristics were obtained using these commands respectively:

```
llength [get_nets -hierarchical]
report_utilization -hierarchical
report_route_status -show_all
report_timing_summary
report_power
```

For estimation of timing, a 10ns clock was assumed:

```
create_clock -period 10.000 -name clk [get_ports clk]
```

And for power utilisation the input switching rate was prescribed, (hence doing a vectorless analysis):

```
set_switching_activity -toggle_rate 2.0 -static_probability 0.95 [get_ports in_valid]
set_switching_activity -toggle_rate 50.0 -static_probability 0.5 [get_ports {din[*]}]
set_switching_activity -toggle_rate 0.0 -static_probability 0.0 [get_ports rst]
```

With this constraints, tool reports medium confidence in power estimation.

The **Artix-7 (part no: xc7a100tcs9324-1)** FPGA was used as reference FPGA on which all synthesis and implementation tasks was done.

2.2 Cooley-Tukey: Radix-2 DIT

Cooley-Tukey algorithm re-expresses the Discrete Fourier Transform (DFT) of an arbitrary composite size $N = N_1 N_2$ in terms of N_1 smaller DFTs of sizes N_2 , recursively, to reduce the computation time to $\mathcal{O}(N \log N)$ for highly composite N . Because of the algorithm's importance, specific variants and implementation styles have become known by their own names, such as Radix-2, Radix-4, mixed-radix, etc. Because the Cooley-Tukey algorithm breaks the DFT into smaller DFTs, it can be combined arbitrarily with any other algorithm for the DFT.

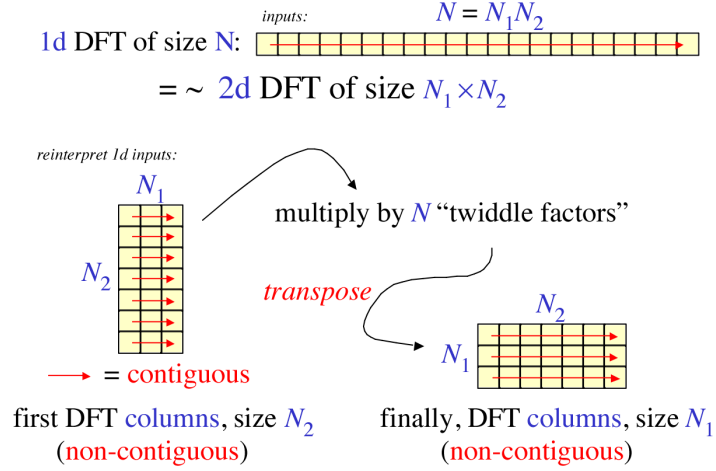


Figure 2.1: Steps involved in Cooley-Tukey algorithm

A radix-2 Decimation in Time (DIT) FFT is the simplest and most common form of the Cooley-Tukey algorithm. Radix-2 DIT divides a DFT of size N into two interleaved DFTs of size $N/2$ with each recursive stage. Hence the overall time complexity of algorithm is $\mathcal{O}(N \log N)$. The following is the derivation of radix-2 DIT:

DFT of sequence x_n is defined by:

$$X_k = \sum_{n=0}^{N-1} x_n \exp\left(-j \frac{2\pi}{N} nk\right)$$

The radix-2 DIT algorithm rearranges the DFT of the function x_n into two parts: a sum over the even-numbered indices $n = 2m$ and a sum over the odd-numbered indices $n = 2m + 1$:

$$\begin{aligned} X_k &= \sum_{m=0}^{N/2-1} x_{2m} \exp\left(-j \frac{2\pi}{N} (2m)k\right) + \sum_{m=0}^{N/2-1} x_{2m+1} \exp\left(-j \frac{2\pi}{N} (2m+1)k\right) \\ &= \underbrace{\sum_{m=0}^{N/2-1} x_{2m} \exp\left(-j \frac{2\pi}{N} mk\right)}_{\text{DFT of even-indexed part of } x_n} + \exp\left(-j \frac{2\pi}{N} k\right) \underbrace{\sum_{m=0}^{N/2-1} x_{2m+1} \exp\left(-j \frac{2\pi}{N} mk\right)}_{\text{DFT of odd-indexed part of } x_n} \\ &= E_k + \exp\left(-j \frac{2\pi}{N} k\right) O_k \end{aligned}$$

Using the periodicity of the complex exponential, $X_{k+\frac{N}{2}}$ is also obtained from E_k and O_k as:

$$X_{k+\frac{N}{2}} = E_k - \exp\left(-j \frac{2\pi}{N} k\right) O_k$$

We can write X_k and $X_{k+\frac{N}{2}}$ as:

$$X_k = E_k + \exp\left(-j\frac{2\pi}{N}k\right)O_k$$

$$X_{k+\frac{N}{2}} = E_k - \exp\left(-j\frac{2\pi}{N}k\right)O_k$$

This result, expressing the DFT of length N recursively in terms of two DFTs of size $N/2$, is the core of the radix-2 DIT FFT. The algorithm gains its speed by re-using the results of intermediate computations to compute multiple DFT outputs. The final outputs are obtained by combination of E_k , O_k and $\exp(-j2\pi k/N)$ (called “twiddle factor” or phase rotation factor) which is simply a size-2 DFT (also called “Butterfly”). The operation can be expressed as a signal flow graph shown in Figure 2.2

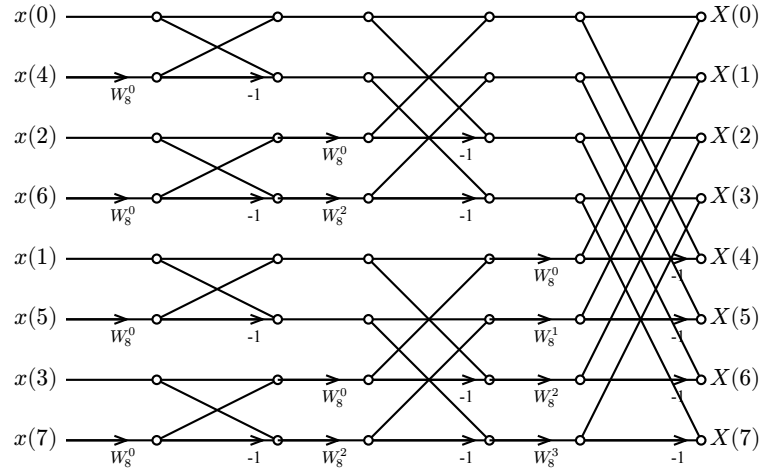


Figure 2.2: Signal flow graph of 8-point radix-2 DIT-FFT

In the software, assuming we have array of input x , the computation begins with reordering the array by swapping element at index i with element at an index which is bit reverse of i . Then subsequently performing radix-2 recursively.

2.2.1 Implementation

The implementation accepts a *fixed point* complex number (given through 2 separate buses for real and imaginary part) and returns computed DFT numbers through another 2 separate buses. This is a burst architecture, hence it waits for all inputs to arrive before starting the computation.

The operation described below:

1. A reset pulse is applied
2. Start signal is issued which starts a counter inside the module.
3. `input_valid` signal is applied and input is passed in each positive edge of clock.
4. After N clock cycle, when all input are be given, `input_valid` is reset and module starts computing

5. After $N \log_2(N)/2$ clock cycles the output will be available
6. To read output, output read enable is raised and read address is passed. In the next clock cycle the output will be obtained.

The whole process takes $N + N \log_2(N)/2 + 1$ clock cycles from passing first input to getting first output: N cycles for applying input and getting output and $N \log_2(N)/2 + 1$ cycles for computation.

The design is fully parametric – the number of points and data width can be changed. The twiddle factor is generated using a python script. Design is limited in terms of throughput and latency and is not useful for high performance use cases. In theory area wise this should be most area efficient design, however due to multiple access to the memory, the synthesizer couldn't infer a BRAM, instead it went for flipflops. As a result the design is not scalable to very extend.

The implementation is by no means efficient for practical use, but serves as a base upon which other algorithms can be implemented and evaluated.

About FSM:

When a start signal is sent, the internal counter starts running. Based on the value of counter address to the addresses of relevant data – inputs to butterfly and twiddle factor – is generated by address generator. A portion the counter is bit reversed to produce an address that will be used to store the input. The computing modules and FSM all also run in parallel. The FSM has 4 states:

- IDLE (not doing any processing)
- LOAD (loading input to internal memory)
- COMPUTE (computing the results and storing it in the memory)
- DONE: All computations are done and ready for next cycle.

The operation of FSM is summarized in Figure 2.3.

After the output is ready, it can be read by raising `output_read_en` and passing the address to be read in `output_read_addr`.

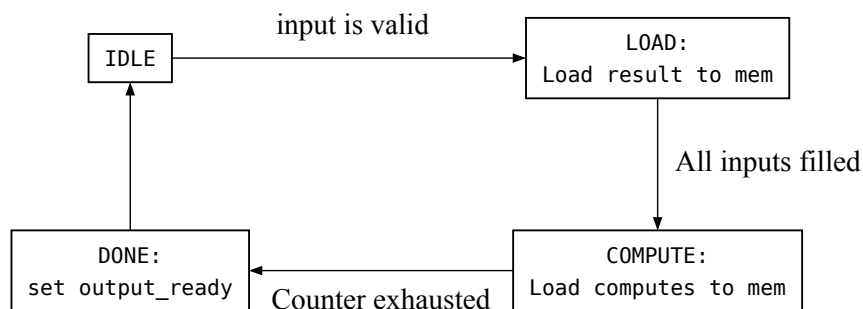


Figure 2.3: Working of FSM in Radix-2 DIT FFT

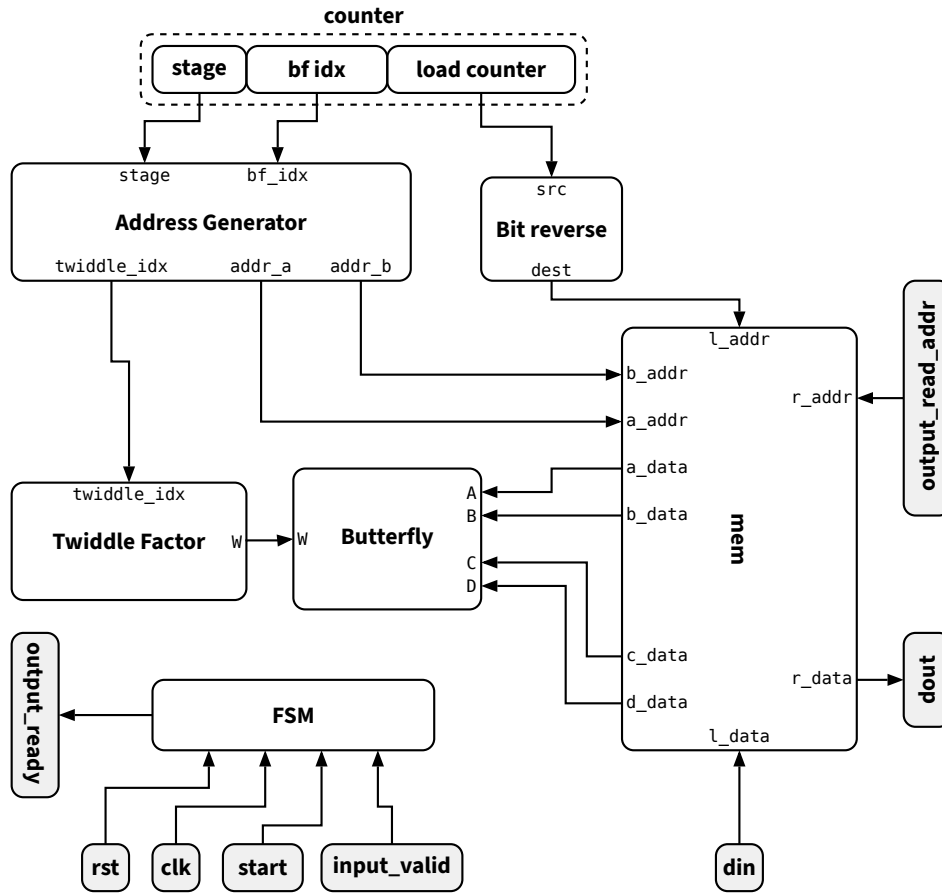


Figure 2.4: Block diagram of Radix-2 DIT FFT implementation

2.2.2 Simulation

Here 4 point DFT is computed in which input is real valued ramp signal. In the Figure 2.5, the output is observed at dout_re and dout_im. The results were verified using *GNU Octave*. From this we can notice that there is a $N + \frac{N}{2} \log_2(N) + 1$ latency to get first output (one additional clock cycle for shift register at output)

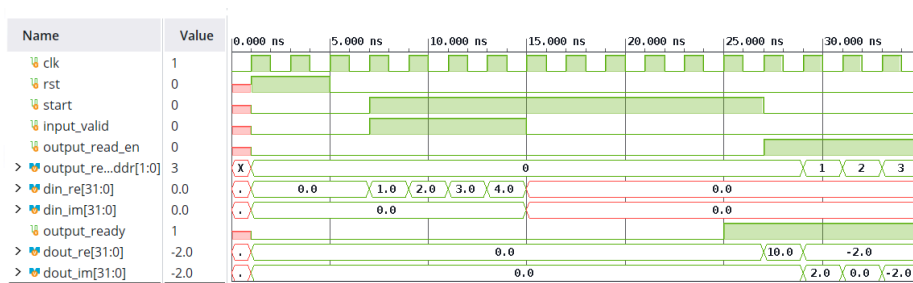


Figure 2.5: Timing diagram of output of 4 point Radix-2 DIT-FFT

2.3 Radix-2 Multi Path Delay Commutator (R2MDC)

This algorithm is based on the Decimation in Frequency (DIF) method. In DIF, the inputs are applied in order, while output obtained will be in bit-reversed form. This allows the input to be fed sequentially. The DIF still gives similar signal flow graph as that of DIT. However the final output has particular pattern that can be used to transform this parallel structure to a pipelined multistaged architecture.

The usual structure of DIF in the case where $N = 4$ is shown in Figure 2.6. Notice that in stage 0, k th and $(k + \frac{N}{2})$ th inputs are required (x_0, x_2 and x_1, x_3). If the data is fed sequentially then one can store first 2 data into a shift register and pop out the value afterwards. This gives the input required for butterfly in the next stage. So the corresponding outputs in each stages are: x'_0, x'_2 and x'_1, x'_3 . But the next stage needs pairs x'_0, x'_1 and x'_2, x'_3 . So twiddle multiplied output of this stage needs to be stored for 1 cycle. Which will satisfy condition for next stage.

In general at stage $s(= 0, 1, \dots, S$ where $S = \lceil \log_2(N) \rceil$ = number of stages) needs shift register of size 2^{S-s} in upper input arm of butterfly and 2^{S-s-1} shift register after the twiddle path. For last stage, multipliers and commutators are not needed. For higher point FFT, design can be scaled easily by cascading more and more R2MDC stages.

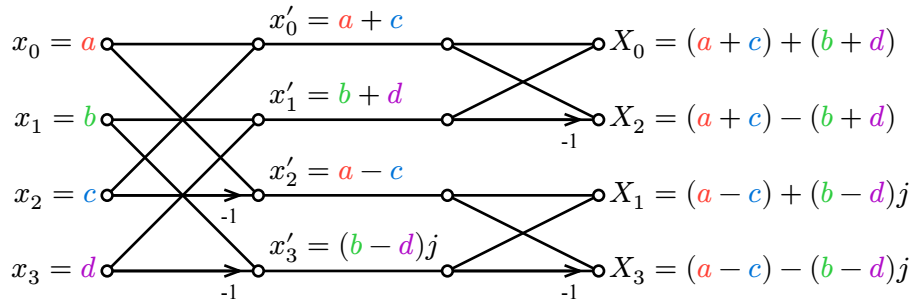


Figure 2.6: Signal flow graph of 4-point radix-2 DIF-FFT

2.3.1 Implementation

The design is fully parametric, in which number of stages, and data width can be changed. The design consists of a controller block that sets the starts the global counter and produces control signals. The outputs out0 and out1 are passed to cascade of R2MDC Stage blocks as shown in Figure 2.8. Each R2MDC stage is a pipelined design consisting of a butterfly unit, twiddle ROM and commutator. Butterfly here only computes sum and difference of incoming inputs, the multiplication with twiddle factor is done using a pipelined multiplier. The required twiddle factors are computed during synthesis and are stored in a LUT. The control signals are also delayed by the same amount as that of data lines.

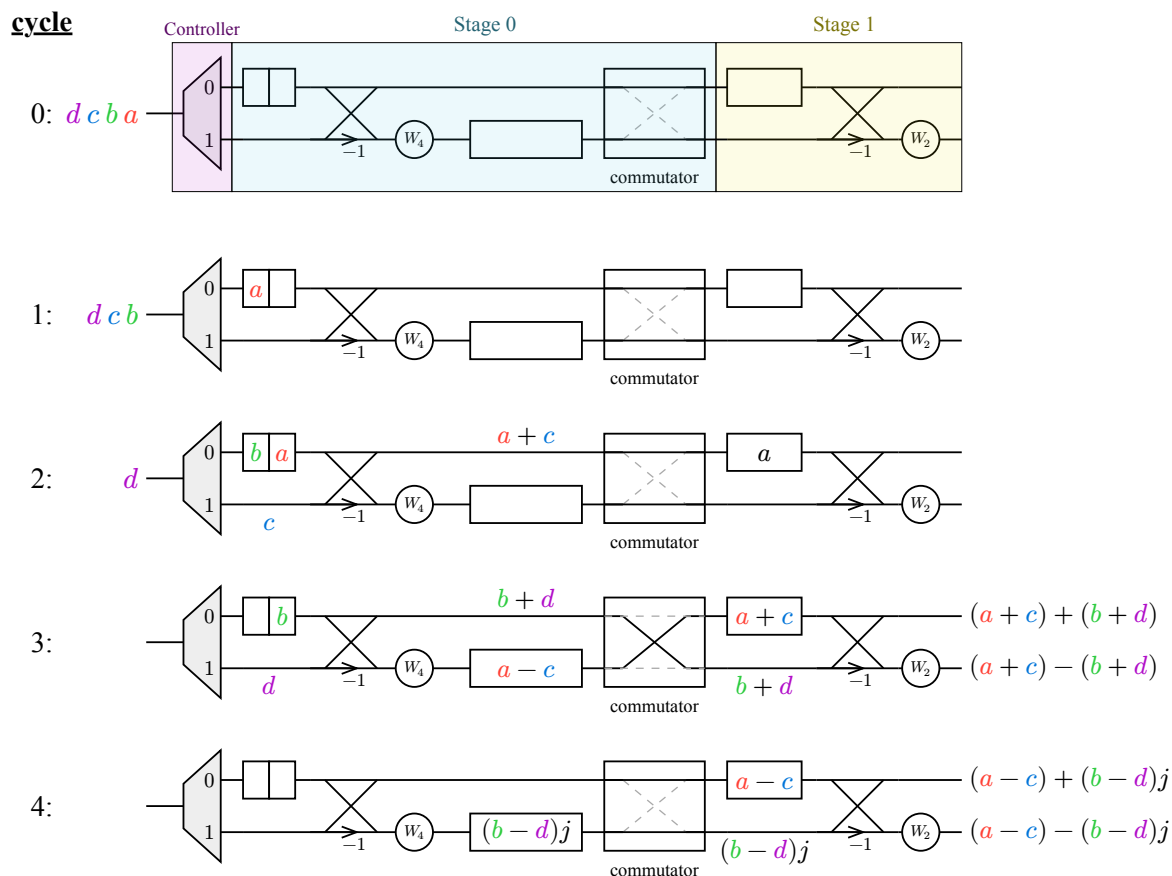


Figure 2.7: Steps in R2MDC algorithm with $N = 4$ case

In each stage the size of delay element reduces by half. No delays, multipliers and commutators are required in final stage. The final output is obtained at out0 and out1 corresponding to bit reversed indexed $X[k]$ and $X[k + N/2]$. The control signals valid and counter are synchronously passed from one stage to another, using delay. As shown in Figure 2.9, two time unit delay is used for pipelining: one for splitting addition and multiplication in butterfly and another one for pipelining complex multiplication in multiplier.

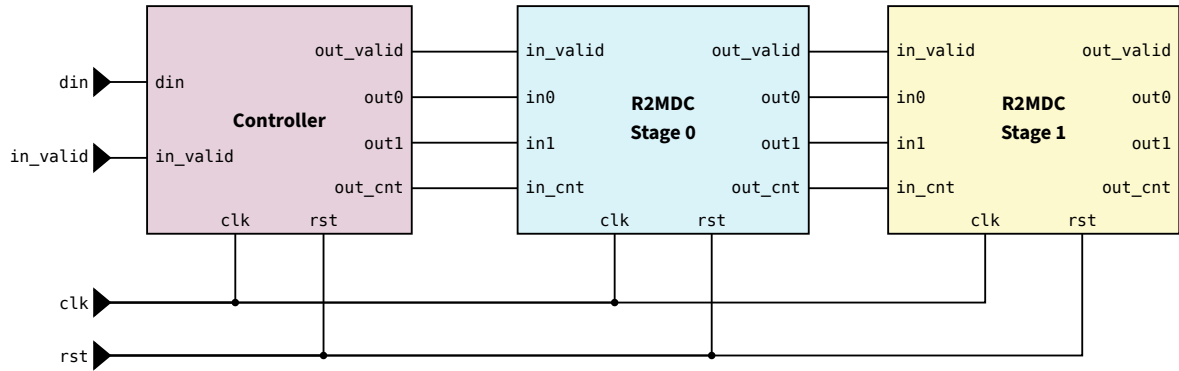


Figure 2.8: Schematic of R2MDC implementation

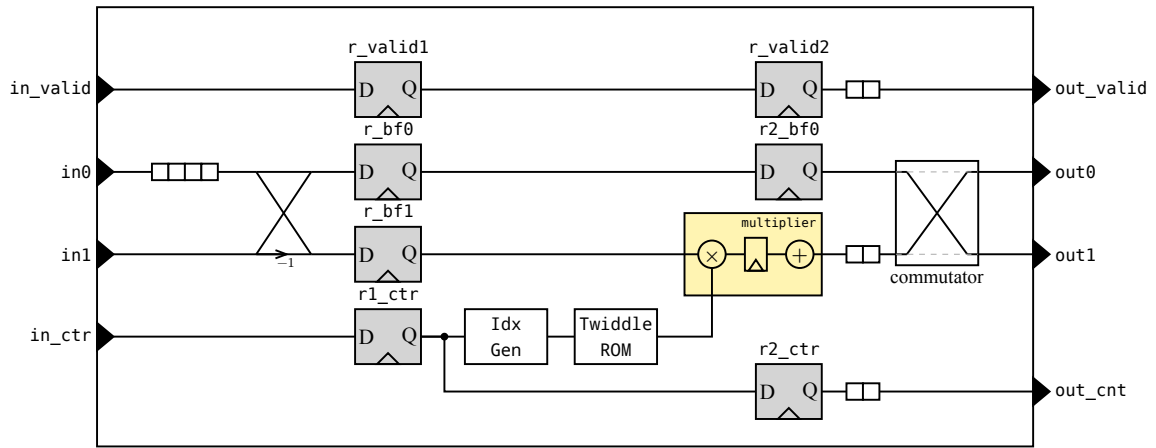


Figure 2.9: Inside a single R2MDC Stage. All modules with wedge is clocked

2.3.2 Simulation result

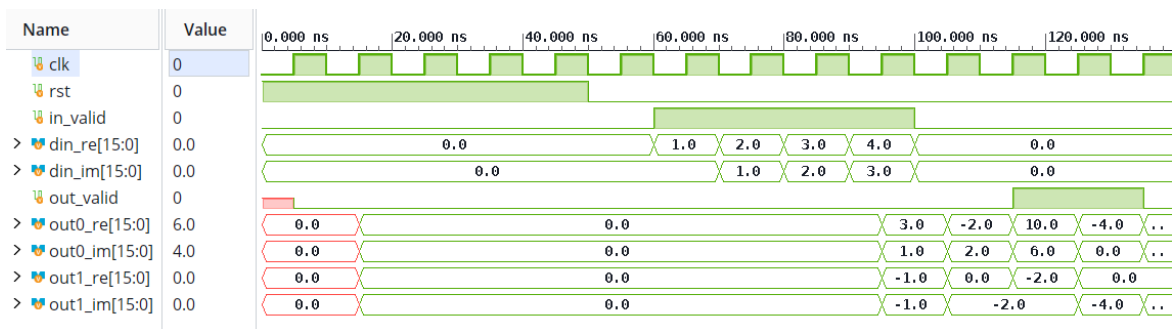


Figure 2.10: Simulation result for 4 point complex test signal

2 outputs are obtained at same clock cycle. Hence it takes $\log_2(N)$ clock cycles to get all outputs after 1st input has been inserted.

2.4 Radix-2 Single Path Delay Commutator (R2SDC)

Single path Delay Commutator is a different pipelined implementation of DIF algorithm. Unlike MDC, SDC produces single output per cycle and uses fewer number of shift registers, at the expense of higher latency and higher complexity of stage controllers. The whole operation is described in Figure 2.13. A overview is given below:

1. In first $N/2$ clock cycles data coming is shifted to shift registers
2. In coming cycles, $x[n]$ and $x[n + N/2]$ datas are available. The butterfly computes the sum and difference, in which the difference is shifted back to the same shift register and sum is passed to twiddle factor multiplier.
3. After all inputs are passed (cycles $> N$) the shift register is completely full of differences. It shifts it out to twiddle factor directly.

2.4.1 Implementation

The design is similar to that of MDC, with the difference that each stage now has its own counter that creates control signal for that stage.

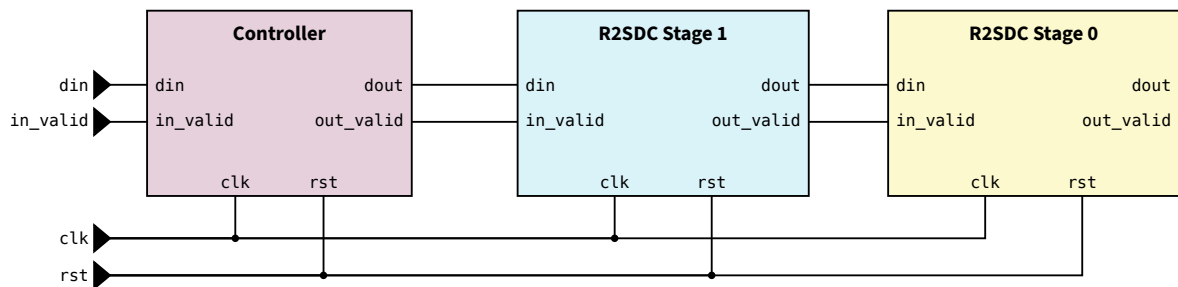


Figure 2.11: Schematic of R2SDC implementation

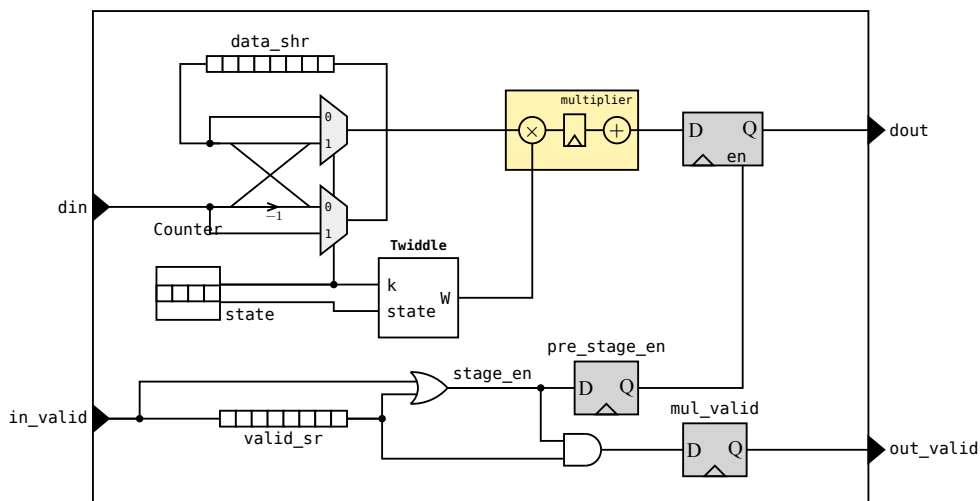


Figure 2.12: Inside a single R2SDC Stage. All modules with wedge is clocked

From this point onwards, the twiddle factors are computed using initial blocks, which is expected to be synthesized to a LUT.

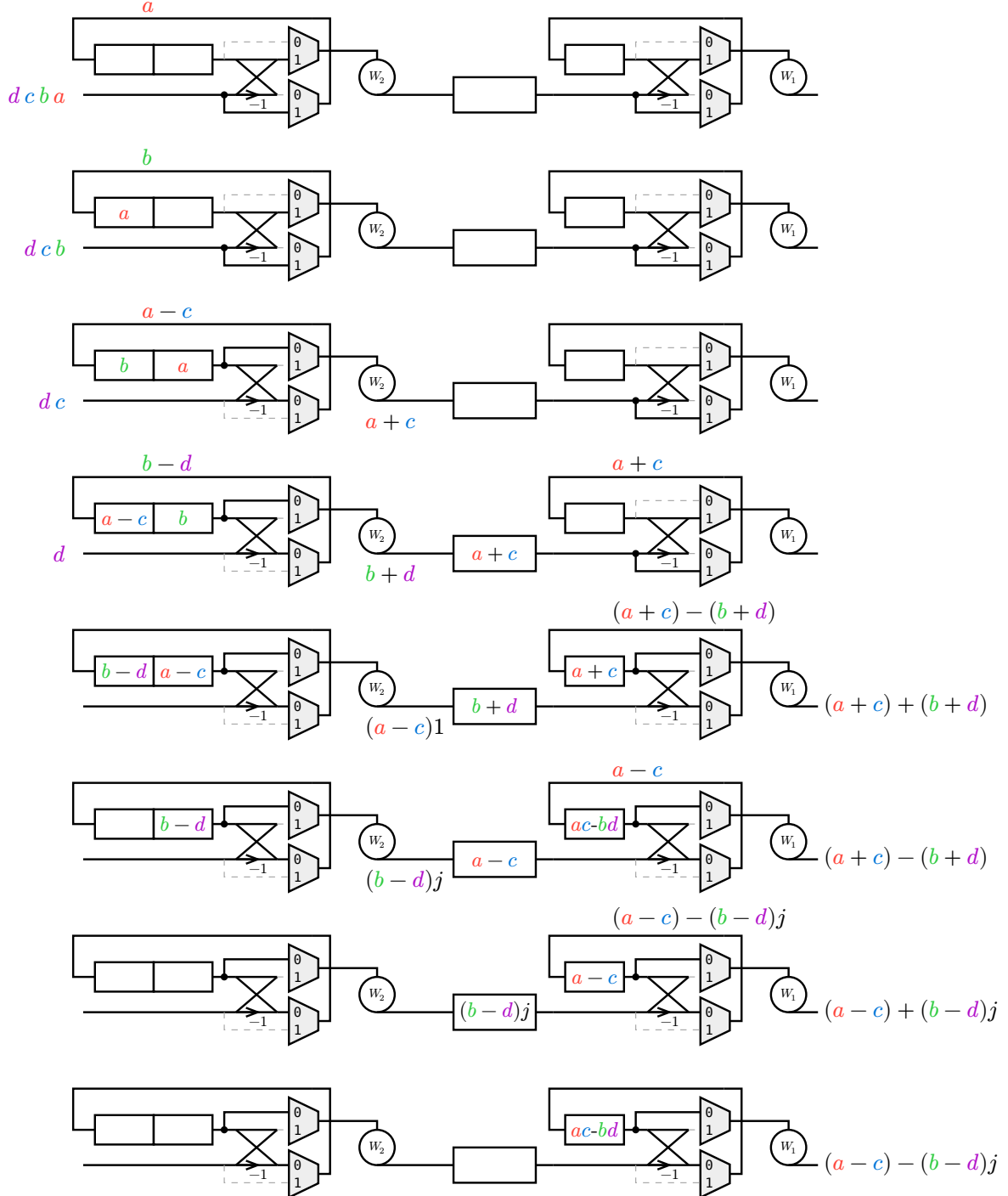


Figure 2.13: Steps involved in a 4 point R2SDC algorithm

2.4.2 Simulation result

The result is similar to that of MDC case. But here due slightly different architecture, the output arrives one clock cycle earlier.

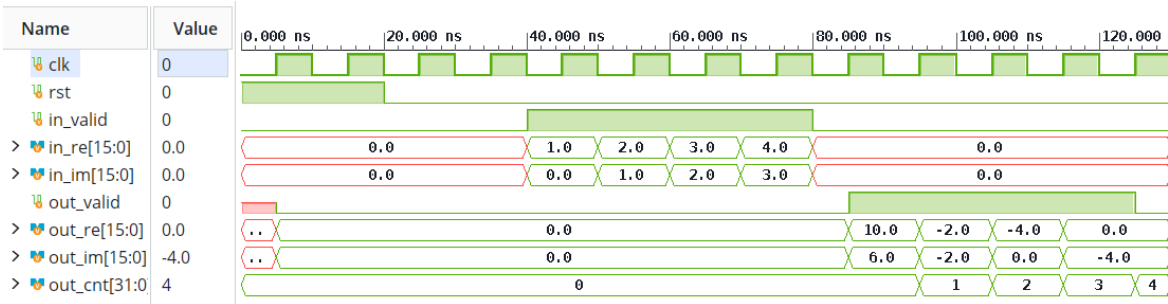


Figure 2.14: Simulation result for 4 point complex test signal

Chapter 3: Specialized methods

3.1 Winograd

Winograd algorithm is actually a fast convolution algorithm which reduces number of multiplications (a strong operation) at the expense of increased number of additions (weak operation). The area requirement for a multiplier is exponential while for an adder it is linear. Hence for large number of bits (≥ 16), this tradeoff results in less silicon area and power. This method finds its use in fully custom designing, but often not used in the settings of FPGA, due to its diminishing returns compared to other methods. However the method is explored here for analysis.

Winograd showed that a prime length DFT can be done as circular convolution, applying input and output Rader index mapping. The method can be derived for any prime size DFT, however the implementation details vary for different lengths. Here implementation of length 5 DFT is shown.

3.1.1 Derivation

The forward transform for 5 point DFT is given by:

$$Y_k = \sum_{n=0}^4 x_n W^{nk}$$

where $W^{nk} = e^{-j2\pi nk/5}$. Because W represents roots of unity of 5, the exponent value nk wraps around every 5 cycle: ie. $W^{nk+5m} = W^{nk}$ for some arbitrary integer m . The operation can be expressed as a matrix multiplication:

$$\begin{bmatrix} Y'(0) \\ Y'(1) \\ Y'(2) \\ Y'(3) \\ Y'(4) \end{bmatrix} = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} W^0 & W^0 & W^0 & W^0 & W^0 \\ W^0 & W^1 & W^2 & W^3 & W^4 \\ W^0 & W^2 & W^4 & W^1 & W^3 \\ W^0 & W^3 & W^1 & W^4 & W^2 \\ W^0 & W^4 & W^3 & W^2 & W^1 \end{bmatrix} \end{matrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

The first row corresponds to DC sum, and the rest of elements can be written as:

$$Y_k = x_0 + Y'(k), \quad k > 0$$

Where $Y'(k)$ is

$$\begin{bmatrix} Y'(1) \\ Y'(2) \\ Y'(3) \\ Y'(4) \end{bmatrix} = \begin{bmatrix} W^1 & W^2 & W^3 & W^4 \\ W^2 & W^4 & W^1 & W^3 \\ W^3 & W^1 & W^4 & W^2 \\ W^4 & W^3 & W^2 & W^1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

The matrix operation can be rearranged in such a way that the matrix becomes Topelitz and eventually Winograd convolution algorithm can be applied. The Rader method gives a permutation scheme for indices to make matrix entries cyclic:

Rader Mapping: When N is a prime number, then a set of non-zero indices $n \in \{1, 2, \dots, N - 1\}$ forms a group under multiplication modulo N . One consequence of numeber theory of such groups is that there exists a integer generator g such that $k = g^p \pmod{N}$ for any non-zero index k and for a unique $p \in \{0, 1, \dots, N - 2\}$ (forming a bijection from p to non-zero k). Similarly $n = g^{-q} \pmod{N}$ for any non-zero index n and for unique $q \in \{0, 1, \dots, N - 2\}$ also forms a bijection (where negative exponent denotes multiplicative inverse of $g^q \pmod{N}$). This means that we can write Y' using new indices p and q as:

$$Y'(g^p) = \sum_{q=0}^{N-2} x(g^{-q}) W^{g^{p-q}}$$

The summation above is identical to cyclic convolution of two sequences $x(g^{-q})$ and $W^{g^{p-q}}$ of length $N - 1$. For the case where $N = 5$, the generator is 2:

$$Y'(2^p) = \sum_{q=0}^5 x(2^{-q}) W^{2^{p-q}}$$

Corresponding matrix form is:

$$\begin{bmatrix} Y'(1) \\ Y'(2) \\ Y'(4) \\ Y'(3) \end{bmatrix} = \begin{bmatrix} W^1 & W^3 & W^4 & W^2 \\ W^2 & W^1 & W^3 & W^4 \\ W^4 & W^2 & W^1 & W^3 \\ W^3 & W^4 & W^2 & W^1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_3 \\ x_4 \\ x_2 \end{bmatrix}$$

Now the operation has become a length-4 cyclic convolution of sequences

$$\begin{aligned} a_q &= x(2^{-q}) = \{x_1, x_3, x_4, x_2\} \\ b_{p-q} &= W^{2^{p-q}} = \{W^1, W^3, W^4, W^2\} \\ Y'(2^p) &= \sum_{q=0}^3 a_q \cdot b_{(p-q) \pmod{4}} \end{aligned}$$

Now we apply Winograd Fast convolution algorithm. For that we are representing this sequence as polynomials with complex coefficients:

$$x'(z) = x_1 + x_3 z + x_4 z^2 + x_2 z^3 \quad W'(z) = W_1 + W_3 z + W_4 z^2 + W_2 z^3$$

The circular convolution is given by: $Y'(z) = x'(z)W'(z) \pmod{z^4 - 1}$. The required values are coefficients of $Y'(z)$

Winograd algorithm:

1. Choose a polynomial $m(z)$ with degree higher than the degree of $W'(z)x'(z)$ and factor it into $k + 1$ relatively prime polynomials with real coefficients, i.e., $m(z) = m^{(0)}m^{(1)}...m^{(k)}$
2. Let $M^{(i)} = m/m^{(i)}$ and use Euclidean GCD algorithm to solve following Bezout's identity and find $N^{(i)}$:

$$m^{(i)}n^{(i)} + M^{(i)}N^{(i)} = 1$$

3. Compute:

$$w^{(i)} = W'(z) \pmod{m^{(i)}} \quad \text{Eq (3.1)}$$

$$x^{(i)} = x'(z) \pmod{m^{(i)}} \quad \text{Eq (3.2)}$$

$$y^{(i)} = w^{(i)}x^{(i)} \pmod{m^{(i)}} \quad \text{Eq (3.3)}$$

for $i = 0, 1, \dots, k$.

4. Compute $Y'(z)$ using:

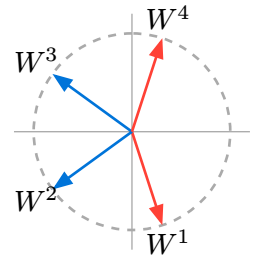
$$Y'(z) = \left[\sum_{i=0}^k y^{(i)} N^{(i)} M^{(i)} \right] \pmod{m(z)} \quad \text{Eq (3.4)}$$

Here $m(z) = z^4 - 1 = (z - 1)(z + 1)(z^2 + 1)$. Required factors are:

i	$m^{(i)}$	$M^{(i)}$	$x^{(i)}$	$w^{(i)}$	N_i
0	$z - 1$	$z^3 + z^2 + z + 1$	$x_1 + x_3 + x_4 + x_2$	$W^1 + W^3 + W^4 + W^2$	$1/4$
1	$z + 1$	$z^3 - z^2 + z - 1$	$x_1 - x_3 + x_4 - x_2$	$W^1 - W^3 + W^4 - W^2$	$-1/4$
2	$z^2 + 1$	$z^2 - 1$	$x_1 - x_4 + z(x_3 - x_2)$	$W^1 - W^4 + z(W^3 - W^2)$	$-1/2$

We initialize some intermediate values:

$$\begin{aligned} c_{00} &= x_1 + x_4 & d_{00} &= W^1 + W^4 = 2 \cos(2\pi/5) \\ c_{01} &= x_3 + x_2 & d_{01} &= W^3 + W^2 = 2 \cos(4\pi/5) \\ c_{20} &= x_1 - x_4 & d_{20} &= W^1 - W^4 = 2 \sin(2\pi/5)j \\ c_{21} &= x_3 - x_2 & d_{21} &= W^3 - W^2 = 2 \sin(4\pi/5)j \end{aligned}$$



and rewrite $x^{(i)}$ and $w^{(i)}$ as :

$$\begin{aligned} x^{(0)} &= c_{00} + c_{01} & w^{(0)} &= d_{00} + d_{01} = -1 \\ x^{(1)} &= c_{00} - c_{01} & w^{(1)} &= d_{00} - d_{01} = 2.236068 \\ x^{(1)} &= c_{20} - c_{21}z & w^{(1)} &= d_{20} - d_{21}z = j1.902113 - j1.175570z \end{aligned}$$

$y^{(i)}$ are computed using Equation Eq (3.3):

$$\begin{aligned} y^{(0)} &= -(c_{00} + c_{01}) &= y_0 \\ y^{(1)} &= (c_{00} - c_{01})2.236068 &= y_1 \\ y^{(2)} &= c_{20}d_{20} - c_{21}d_{21} - z(c_{20}d_{21} + c_{21}d_{20}) \\ &= y_{20} - zy_{21} \end{aligned}$$

Finally using Equation Eq (3.4):

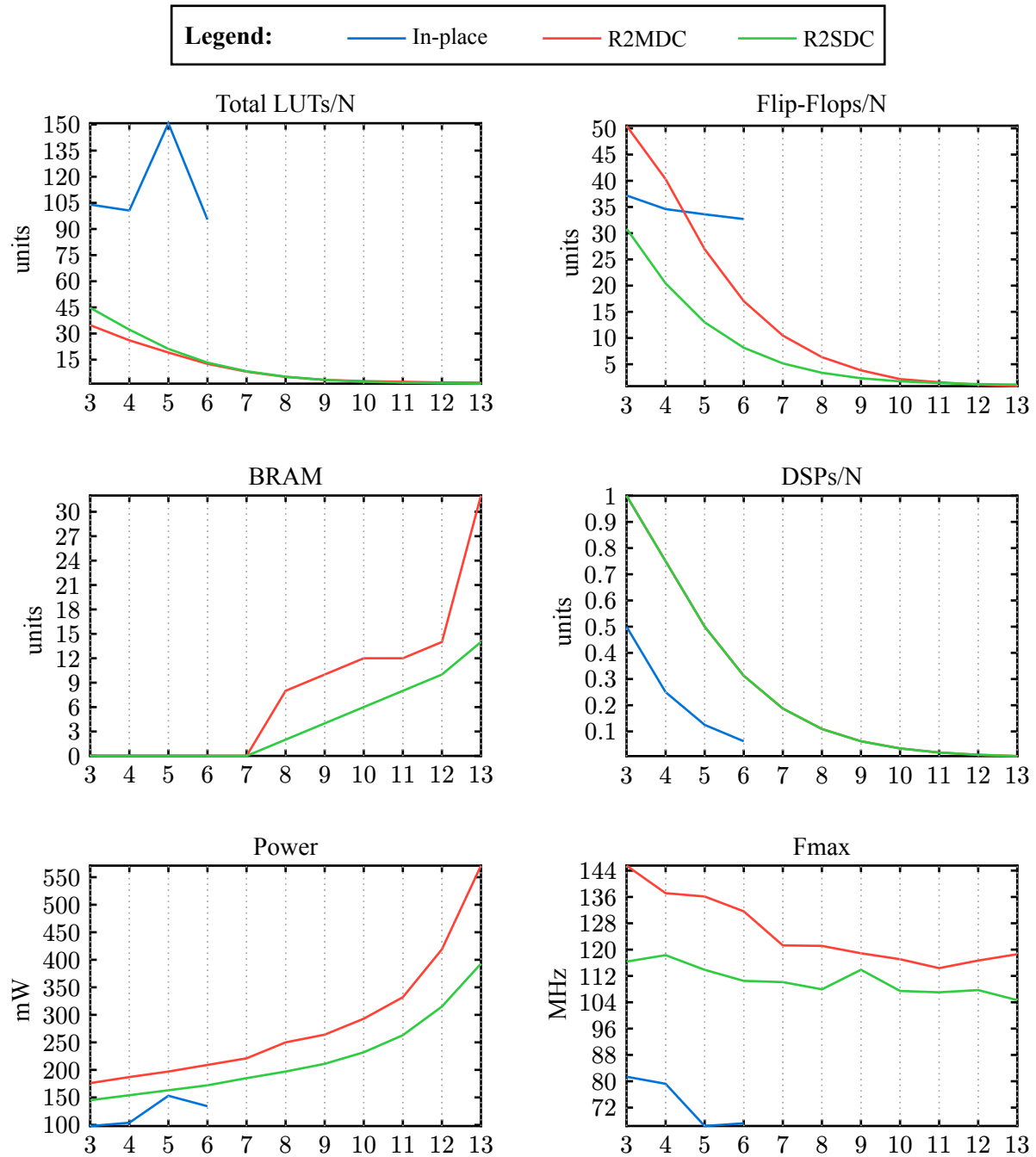
$$\begin{aligned} Y'(z) &= \frac{y_0}{4}(z^3 + z^2 + z + 1) - \frac{y_1}{4}(z^3 - z^2 + z - 1) - \frac{1}{2}(y_{20} - zy_{21})(z^2 - 1) \\ &= z^3 \left[\frac{y_0}{4} - \frac{y_1}{4} + 0 + \frac{y_{21}}{2} \right] + z^2 \left[\frac{y_0}{4} + \frac{y_1}{4} - \frac{y_{20}}{2} + 0 \right] + \\ &\quad z \left[\frac{y_0}{4} - \frac{y_1}{4} + 0 - \frac{y_{21}}{2} \right] + \left[\frac{y_0}{4} + \frac{y_1}{4} + \frac{y_{20}}{2} + 0 \right] \end{aligned}$$

Hence corresponding coefficients are:

$$\begin{aligned} \begin{bmatrix} Y'(1) \\ Y'(2) \\ Y'(4) \\ Y'(3) \end{bmatrix} &= \begin{bmatrix} 1 & -1 & 0 & 1 \\ 1 & 1 & -1 & 0 \\ 1 & -1 & 0 & -1 \\ 1 & 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} -(c_{00} + c_{01})/4 \\ (c_{00} - c_{01})2.236068/4 \\ (c_{20}d_{20} - c_{21}d_{21})/2 \\ (c_{20}d_{21} + c_{21}d_{20})/2 \end{bmatrix} \\ \begin{bmatrix} Y'(1) \\ Y'(2) \\ Y'(4) \\ Y'(3) \end{bmatrix} &= \begin{bmatrix} 1 & -1 & 0 & 1 \\ 1 & 1 & -1 & 0 \\ 1 & -1 & 0 & -1 \\ 1 & 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} y_0/4 \\ y_1/4 \\ y_{20}/2 \\ y_{21}/2 \end{bmatrix} \end{aligned}$$

Chapter 4: Comparison

All datas have been plotted here for quick comparison:



Analysis and Implementation of FFT Algorithms

The throughput is not plotted here as it closely matches with $F_{\max}$. From this we its clear that inplace algorithm performs worse in all cases except at power consumption. Also MDC architecture is preferred in high throughput requirements (such as radar, SDR, etc.). With both pipelined architectures, the synthesis tool was able to infer BRAM blocks past a certain number of transform size.